



2.11. Spring

Привет!

На связи шпаргалка урока 2.11. Spring, в котором вы найдете материалы по теме урока.

Время прочтения: 25 минут.

В шпаргалке к уроку 2.3 уже была дана базовая идея Spring, Spring Boot и Spring Web. Рекомендуем просмотреть ее перед прочтением текущей шпаргалки.

Сегодня мы глубже погрузимся в возможности Spring и изучим некоторые базовые возможности данного фреймворка, которые встречаются во многих Spring-проектах.

Способы создания бинов

Аннотации

@Component

@Service

@Repository

@Controller

@RestController

Java-config

@Configuration

@Bean

XML/Groovy-config

«Выбор» необходимого бина

Способы инъекции бинов

Циклическая зависимость бинов

Скоупы бинов

Жизненный цикл бина
Основной конфигурационный файл
Данные из конфигурационного файла
Зависимость данных из конфигурации
Выполнение методов «по-расписанию»
Jackson
Обработка исключений

Способы создания бинов



Напомним, что в упрощенном виде бинами называются объекты, которыми управляет (создает, удаляет, добавляет в ваши классы) Spring.

Мы уже научились создавать бины с помощью аннотаций `@RestController` и `@Service`.

Сегодня изучим все актуальные способы создания бинов.

Аннотации

Этот способ вам уже знаком на примере двух аннотаций выше.

Но это не единственные две аннотации, которые позволяют разметить класс в качестве бина.

`@Component`

Основной аннотацией, размечающей бины, является аннотация `@Component`.

Эта аннотация является родительской, от нее уже пошли унаследовавшие ее `@Controller`, `@RestController`, `@Service` и `@Repository`.

Этой аннотацией помечаются бины, которые не содержат в себе бизнес-логику, но выполняют какую-то функцию и должны быть в нашем приложении.

Одним из популярных «кейсов» может быть необходимость создать объект, который из вашего приложения делает HTTP-запрос к другому приложению или веб-сайту, получая оттуда данные и возвращая их запросившему сервису.

Данный веб-клиент может потребоваться в нескольких местах нашего приложения и, чтобы не привязывать его к конкретному месту, мы выносим

функционал отправки HTTP-запросов в отдельный класс и размечаем его аннотацией `@Component`.

После этого любой из сервисов сможет запросить у Spring нашего клиента стандартным способом (через объявление поля и конструктор) и начать его использовать.

@Service

С этой аннотацией вы уже встречались. Ей помечаются классы, которые выполняют бизнес-логику. Сервисы содержат набор полей, необходимых для выполнения логики. Полями могут быть как другие сервисы, так и независимые компоненты вроде упомянутого выше клиента.

Задача сервиса — получить запрос от контроллера с пришедшими в контроллер параметрами и провести необходимые для выполнения задачи вычисления, затем вернуть результат.

С технической точки зрения `@Service` и `@Component` не отличаются друг от друга.

Никакого дополнительного функционала с точки зрения Spring `@Service` не несет, потому размечать бизнес-классы этой аннотацией является хорошим тоном, а не жесткой необходимостью.

@Repository

Данной аннотацией помечаются классы, которые являются хранилищами неких данных. Вы уже разрабатывали некий гибрид сервиса и репозитория для хранения данных в виде `EmployeeService`. Этот подход был не совсем корректный.

В Spring-приложениях для работы с данными используются именно репозитории.

Стандартный набор их функций: CRUD-операции (create, read, update, delete), т. е. добавление, поиск, обновление и удаление сущностей, которыми репозиторий управляет. В вашем случае это была сущность `Employee`.

Обычно репозитории работают с базой данных, но бывают и случаи, когда сущности хранятся в коллекции.

В вашем случае использовались сначала массивы, потом коллекции, а в дальнейшем проект перейдет на базу данных.

С технической точки зрения `@Repository` от `@Component` отличается дополнительной логикой по работе с исключениями. Поэтому для любых хранилищ данных желательно использовать репозиторий.

@Controller

Данной аннотацией помечаются классы-контроллеры, которые возвращают пользователю веб-страницу или HTML-строку.

В современной разработке практически не используется, так как большинство современных Spring-приложений перешли на коммуникацию «друг с другом» через JSON/XML.

@RestController

Данной аннотацией помечаются классы-контроллеры, которые возвращают пользователю (или приложению) некий объект (или коллекцию объектов) в формате JSON/XML.

С технической точки зрения является «сборной» аннотацией, состоящей из `@Controller` и `@ResponseBody`, вторая из которых позволяет Spring трактовать возвращенный из метода объект в качестве ответа на запрос к контроллеру, переводя его в JSON или XML.

Java-config

В ситуациях, когда вам нужно конфигурировать бин на основе переданных параметров (например, из конфигурации приложения), а не просто создать объект, на помощь приходит Java-config.

Этот вид конфигурации бинов выглядит как обычный Java-класс, где каждый метод создает некий объект, но по факту является конфигурационным файлом и некоторые «концепции», применимые к коду на Java, в этом случае неуместны.

Начнем с применяемых для Java-config аннотаций.

@Configuration

Этой аннотацией помечаются классы, которые являются Java-конфигурациями.

Именно благодаря этой аннотации Spring понимает, что перед ним Java-конфигурация.

@Bean

Этой аннотацией помечаются методы, возвращаемое значение (результат) которых будет помещено в контекст Spring и вы сможете его инджектить в другие бины.

Имя метода, помеченного этой аннотацией, будет использоваться в качестве ID бина, т. е. по аналогии с Map ключом бина.

Выглядит Java-config следующим образом.

Представим, что нам нужно создать объект класса Random, передав ему при создании некое число seed, что будет использоваться в логике генерации рандомных значений.

Так как этот класс является классом из стандартной библиотеки, переписать его код и «захардкодить» внутри наше значение мы не можем.

Написать свой конструктор, устанавливающий стандартное значение, мы не можем тоже.

Следовательно, создать объект нам нужно вручную.

Пусть нашим seed в данном примере будет число 256.

▼ В этом случае Java-конфигурация будет выглядеть следующим образом.

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.util.Random;

@Configuration
public class RandomConfiguration {

    @Bean
    public Random random() {
        return new Random( seed: 256);
    }
}
```

Как видите, мы написали метод random(), где вручную создали объект типа Random, вручную передав в него значение 256.

Имя метода будет использоваться в качестве идентификатора бина и мы всегда сможем написать еще один (или множество) методов, создающих

Random, где изменить seed или вообще оставить его дефолтным (пустой конструктор).

Благодаря этому самому идентификатору мы сможем запросить у Spring необходимый именно нам объект типа Random.

XML/Groovy-config

За время существования Spring было придумано много способов разметки классов для создания бинов.

Еще два способа работали на базе XML-конфига, когда все бины описывались в одном XML-файле.

Способ был очень популярен до появления Spring Boot, который «из коробки» предоставил нам возможность автоконфигурации (он самостоятельно сканирует все пакеты и классы, которые находятся в том же месте, где запускающий класс).

Groovy-конфиг был еще одной вариацией, но популярным не стал.

Эти способы уже скорее устаревшие, и встретить их в новых проектах практически невозможно.

«Выбор» необходимого бина

▼ Вернемся к упомянутой в разделе Java-конфига вариативности бинов одного типа.

```
@Configuration
public class RandomConfiguration {

    @Bean
    public Random random() {
        return new Random( seed: 256);
    }

    @Bean
    public Random defaultRandom() {
        return new Random();
    }
}
```

Выше мы написали конфигурацию, где создаем два объекта типа Random.

При запросе бина этого типа стандартным способом (объявить поле и указать в конструкторе) Spring выдаст ошибку.

▼ Посмотрим на примере кода.

```
@Service
public class RandomService {

    private final Random rand;

    @Contract(pure = true)
    public RandomService(Random rand) {
        this.rand = rand;
    }

    public int getRandomInt() {
        return rand.nextInt();
    }
}
```

Как видите, IDEA сразу подчеркивает переменную красным.

▼ Если навести на красную черту, будет выведено предупреждение, что невозможно подставить бин, т. к. есть два варианта.

```
Could not autowire. There is more than one bean of 'Random' type.
Beans: defaultRandom (RandomConfiguration.java)
       random (RandomConfiguration.java)

Add qualifier Alt+Shift+Enter  More actions... Alt+Enter
```

▼ Если запустить приложение, будет выброшена ошибка инициализации контекста.

```
*****
APPLICATION FAILED TO START
*****

Description:

Parameter 0 of constructor in pro.sky.java.itemstore.service.RandomService required a single bean, but 2 were found:
- random: defined by method 'random' in class path resource [pro/sky/java/itemstore/configuration/RandomConfiguration.class]
- defaultRandom: defined by method 'defaultRandom' in class path resource [pro/sky/java/itemstore/configuration/RandomConfiguration.class]

Action:

Consider marking one of the beans as @Primary, updating the consumer to accept multiple beans, or using @Qualifier to identify the bean that should be consumed
```

В описании мы видим, что есть два подходящих по типу бина, и Spring не может решить за нас, какой конкретно мы хотим получить.

Пути решения два.

Первый и самый лаконичный — переименовать поле.

- ▼ Spring возьмет имя поля и попытается найти по такому идентификатору объект, используя имя поля в качестве ключа.

```
@Service
public class RandomService {

    private final Random random;

    @Contract(pure = true)
    public RandomService(Random random) {
        this.random = random;
    }
}
```

- ▼ Как видите, предупреждение исчезло. Это произошло потому, что наш метод из конфига выше назван random().

```
@Service
public class RandomService {

    private final Random defaultRandom;

    @Contract(pure = true)
    public RandomService(Random defaultRandom) {
        this.defaultRandom = defaultRandom;
    }
}
```

Также проблемы не будет, если назвать поле defaultRandom по имени второго нашего конфигурационного метода.

- ▼ Если вас не очень радует именование поля по идентификатору бина (а иногда идентификаторы бинов могут содержать десятки символов), можно использовать аннотацию `@Qualifier`, которая в качестве параметра принимает ID желаемого бина.

```

@Service
public class RandomService {

    private final Random rand;

    @Contract(pure = true)
    public RandomService(@Qualifier("random") Random rand) {
        this.rand = rand;
    }
}

```

Как видите, это решение тоже сработало.

Но что, если мы хотим пометить какой-то из бинов «основным» и инджектировать его всюду, где не указан `@Qualifier`?

▼ Здесь нам на помощь приходит аннотация `@Primary`.

```

@Bean
@Primary
public Random defaultRandom() {
    return new Random();
}

```

▼ Один из бинов мы пометили `@Primary`. Spring покажет, что это стандартная реализация и во все конфликтные места подставлять нужно ее.

```

@Service
public class RandomService {

    private final Random rand;

    @Contract(pure = true)
    public RandomService(Random rand) {
        this.rand = rand;
    }
}

```

Как видите, ошибки снова нет, т. к. `@Qualifier` отсутствует, в поле `rand` будет подставлен объект, помеченный `@Primary`.

Стоит добавить, что `@Primary` может использоваться не только в Java-конфигурациях, но и в конфигурациях на базе аннотаций.

Способы инъекции бинов



Вы уже знаете, что основным способом инъекции бинов является конструктор.

Этот способ не требует каких-то специфических аннотаций и выглядит в целом так же, как стандартное создание объекта в Java.

Но этот способ не единственный.

Остальные способы не являются рекомендуемыми в большинстве случаев, но в редких ситуациях могут быть полезны.

Всего их два (помимо основного). Это инъекция через сеттер и инъекция через поле.

Начнем с инъекции через поле.

▼ Для инъекций не через конструктор нам потребуется аннотация `@Autowired`.

```
@Service
public class RandomService {

    @Autowired
    private Random random;
```

Этот способ выглядит короче и, возможно, лаконичнее, но есть минусы.

Основными минусами является потеря контроля над созданием объекта.

Вы не можете через конструктор создать невалидный объект, так как все зависимости, необходимые для его работы, подаются в конструктор.

Если вам кажется, что Spring позволяет никогда не создавать объекты вручную, то вы ошибаетесь.

В тестировании, например, создавать объекты с инъекцией через конструктор является крайне удобной практикой, в отличие от инъекции через поля.

Также, как было указано выше, это дополнительная защита от создания неработоспособного объекта.

Есть еще некоторое количество ситуаций, когда инъекция через конструктор будет наиболее удобной.

▼ Теперь рассмотрим инъекции через сеттер.

```
@Service
public class RandomService {

    private Random random;

    @Autowired
    public void setRandom(Random random) {
        this.random = random;
    }
}
```

В данном способе инъекция осуществляется с помощью сеттера.

Это еще один способ, который является вполне допустимым.

В некоторых статьях можно встретить концепцию, что необходимые зависимости следует инджектировать через конструктор, а второстепенные (которые не намертво прибиты к бизнес-логике, вроде логгеров или профайлеров) инджектировать через сеттер.

Соглашаться с этой идеей или нет, вам предстоит разобраться на опыте. На данном этапе вы должны знать, что такой способ инджекции тоже возможен.

Разработчики Spring рекомендуют использовать во всех возможных ситуациях инджекцию через конструктор.

В этом случае к тому же есть плюс по производительности, так как фреймворку не требуется анализировать набор полей или методов класса.

Циклическая зависимость бинов

Обратимся к проблеме, которая периодически возникает при написании приложений с инджекцией через конструктор.

Иногда может сложиться так, что для работы одному сервису требуется другой сервис, а другому сервису требуется наш первый сервис.

Вспомним проект с книгой сотрудников, который вы разрабатываете на втором курсе.

Представим ситуацию, когда развитие вашего проекта подразумевает, например, добавление SalaryService, что будет содержать в себе методы по работе с зарплатами.

- ▼ Воспроизведем ситуацию, когда нам нужно индексировать зарплату сотруднику с самой маленькой ЗП.

```
@Service
public class EmployeeService {

    private final SalaryService salaryService;

    private final Map<String, Employee> employees = new HashMap<>();

    public EmployeeService(SalaryService service) { this.salaryService = service; }

    public void indexSalaryOfEmployeeWithMinSalary(int percent) {
        Employee employee = salaryService.findEmployeeWithMinSalary();
        if (employee != null) {
            salaryService.indexSalary(employee, percent);
        }
    }

    public Collection<Employee> getEmployees() {
        return Collections.unmodifiableCollection(employees.values());
    }
}
```

```

@Service
public class SalaryService {

    private final EmployeeService employeeService;

    @Contract(pure = true)
    public SalaryService(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }

    public void indexSalary( @NotNull Employee employee, int percent) {
        int currentSalary = employee.getSalary();
        float income = currentSalary * (percent / 100f);
        employee.setSalary(Math.round(currentSalary + income));
    }

    public Employee findEmployeeWithMinSalary() {
        return employeeService.getEmployees().stream()
            .min( comparator: Comparator.comparingInt( keyExtractor: Employee::getSalary))
            .orElse( other: null);
    }
}

```

Абстрагируемся от того, что архитектура не совсем корректная.

Возможно, метод должен быть написан полностью в SalaryService, а не в EmployeeService, но наша текущая задача не в этом.

Как видите, образовалась циклическая зависимость, т. е. один сервис ожидает получить в конструктор бин другого, второй сервис ожидает получить в конструктор бин первого.

▼ При запуске появится ошибка инициализации контекста.

```

APPLICATION FAILED TO START
*****
Description:
The dependencies of some of the beans in the application context form a cycle:

EmployeeService defined in file [D:\Projects\litem-store\build\classes\java\main\pro\litemstore\EmployeeService.class]
↳ SalaryService defined in file [D:\Projects\litem-store\build\classes\java\main\pro\litemstore\SalaryService.class]
↳ EmployeeService

Action:
Relying upon circular references is discouraged and they are prohibited by default. Update your application to remove the dependency cycle between beans. As a last resort, it may be possible to break the cycle automatically by setting spring.main.allow-circular-references to true.

```

В ошибке написано, что два класса ссылаются друг на друга и что построить граф зависимостей, а затем сконструировать объекты невозможно.

Именно в такой ситуации необходимо использовать инджекцию через сеттер.

В этом случае оба сервиса будут созданы, а уже потом будут вызваны необходимые сеттеры.

Благодаря такой последовательности никакой циклической зависимости не случится.

Но, конечно, циклическая зависимость в большинстве ситуаций является маркером не очень хорошей архитектуры приложения.

Скоупы бинов



Скоупом называется политика, по которой Spring будет хранить и инджектировать бины.

Их всего 6, 2 из которых являются основными, а еще 4 являются специальными для веб-приложений.

Эти скоупы:



Singleton — является стандартным и используется по умолчанию. Подразумевает, что объект будет создан в единственном экземпляре. На все запросы будет выдан один и тот же объект.



Prototype — является вторым стандартным скоупом. Полная противоположность синглтона. На каждый запрос нового бина будет создан новый объект.



Request — является веб-скоупом. Новый объект создается на каждое обращение пользователя к контроллеру.



Session — является веб-скоупом. Новый объект создается на время существования сессии и закрепляется за конкретным браузером. Все запросы с определенного браузера в течение определенного промежутка времени будут работать с одним объектом. При обращении с другого браузера или через длительный период будет открыта новая сессия и создан новый объект.



Websocket — является веб-скоупом. Привязан к сессии веб-сокета. Работает примерно так же, как предыдущий скоуп, но привязывается к устройству, а не к браузеру.



Application — является веб-скоупом. В общем и целом работает так же, как Singleton, но привязан не к контексту приложения, а к веб-контексту.

Веб-скоупы используются достаточно редко. Последние два не используются практически никогда. Самыми популярными являются singleton и prototype.

▼ Прописать скоуп бина можно с помощью аннотации `@Scope`, указав в скобках его название. Если скоуп не прописан, используется singleton.

```
@Service
@Scope("prototype")
public class EmployeeService {
```

▼ Веб-скоуп бина можно прописать как с помощью аннотации `@Scope`, так и с помощью специальных аннотаций вида `@RequestScope` или `@SessionScope`.

```
@Bean
@SessionScope
public Bucket bucket() { return new Bucket(); }
```

Обратите внимание, что скоуп можно прописывать при любом способе конфигурации (аннотации или Java-конфигурация).

Жизненный цикл бина

У бинов существует жизненный цикл.

Бины, будучи созданными, имеют несколько этапов своей «жизни».

Сразу после создания происходит этап инициализации, где Spring позволяет провести некоторые первичные манипуляции с объектом. Например, заполнить

его поля с коллекциями какими-то значениями.

Еще один из этапов наступает перед удалением объекта. Этот этап позволяет произвести закрытие ресурсов (если читались файлы, например).

Разберем этап, который возникает после создания объекта.

На этом этапе мы можем вызвать необходимый нам код. Этот код отработает в роли второго конструктора.

Подобный код находится в init-методе.

Чтобы объявить свой init-метод, мы должны создать метод без параметров и пометить его аннотацией `@PostConstruct`.

- ▼ В тот момент, когда наш бин будет создан, Spring проверит существование init-метода и вызовет его при необходимости.

```
@Service
public class TestService {

    @PostConstruct
    public void init() {
        System.out.println("Service is here.");
    }
}
```

- ▼ И теперь при старте приложения мы увидим в консоли информацию, что сервис был создан.

```
2021-11-25 14:51:08.818 INFO 25060 --- [           main] w.s.c.ServletWebServerApplicationContext
Service is here.
2021-11-25 14:51:08.990 INFO 25060 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer
```

Стоит добавить, что в момент вызова init-метода объект уже полностью создан и функционален.

Теперь перейдем к этапу, который происходит перед удалением объекта.

- ▼ На этом этапе вызывается destroy-метод. В нем обычно происходит закрытие ресурсов или выводится какая-то отладочная информация.

```
@PreDestroy
public void destroy() {
    System.out.println("Service is killed.");
}
```

▼ Протестируем наш сервис, пометив его `@RequestScope`.

```
@Service
@RequestScope
public class TestService {

    @PostConstruct
    public void init() { System.out.println("Service is here."); }

    public void doSmt() { System.out.println("Hello, world!"); }

    @PreDestroy
    public void destroy() { System.out.println("Service is killed."); }
}
```

По задумке при обращении к нашему сервису через браузер мы должны в консоли увидеть 3 сообщения.

▼ Напишем простой контроллер и заинджектим туда наш сервис.

```
@RestController
public class TestController {

    private final TestService testService;

    @Contract(pure = true)
    public TestController(TestService testService) {
        this.testService = testService;
    }

    @GetMapping
    public Object testService() {
        testService.doSmt();
        return null;
    }
}
```

▼ Сначала должно вывести "Service is here", затем "Hello, world!", а затем "Service is killed."

```
2021-11-25 14:55:25.391 INFO 20040 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2021-11-25 14:55:25.391 INFO 20040 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 0 ms
Service is here.
Hello, world!
Service is killed.
```

Как видите, всё отработало корректно.

Благодаря `@RequestScope`, наш сервис просуществовал ровно то время, которое обрабатывался запрос.

Как только мы отправили запрос, сервис был создан, затем был вызван `init`-метод, затем был вызван его метод `doSmth()`, а под конец существования объекта был вызван `destroy`-метод и уничтожение сервиса.

Следует запомнить, что для скоупа `prototype` `destroy`-метод не вызывается.

Это связано с тем, что `prototype`-бины не помещаются в контекст спринга и, соответственно, Spring после создания объекта забывает, что эти объекты когда-то существовали.

Основной конфигурационный файл

Мы закончили с бинами и теперь переходим к другим фичам.

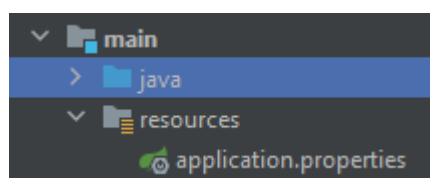
Каждый проект на базе Spring имеет конфигурационный файл, который генерируется IntelliJ или IDEA и используется для хранения конфигураций.



В реальных проектах при запуске проекта на этот файл могут подкладываться в папку с JAR-архивом приложения, и Spring, проанализировав файлы «вокруг» архива, подтягивает конфигурацию из рядом лежащего файла без необходимости пересобирать архив.

Этот файл называется `application`. Он может быть как в формате `.properties`, так и в `.yaml`.

▼ Файл находится внутри папки `resources` вашего приложения.



После генерации данный файл пустой, но вы можете прописать в него множество конфигурационной информации.

Например, можете прописать веб-путь к приложению и порт, на котором он будет поднят.

▼ Например.

```
server.servlet.context-path=/store  
server.port=9090
```

Эти строки скажут Spring, что приложение должно работать не просто на localhost:8080, а на localhost:9090/store, а уже далее Spring будет анализировать путь и передавать запросы определенным конструкторам.

При попытке обратиться по адресу localhost:9090 без частицы /store вы получите ошибку.

Этот context-path позволяет на одном порту разворачивать несколько приложений.

Обычно в данном файле указываются пути к базам данных, веб-адреса и порты к другим приложениям, логины пароли и т. д.

Данные из конфигурационного файла

Конфигурационный файл позволяет, помимо хранения служебных данных, содержать в нем и свои данные, которые затем можно подставить в нужные нам переменные.

Для этого используется аннотация @Value.

▼ Добавим в application.properties следующую строку.

```
valueFromConfigurationFile=Hello from configuration, world!
```

▼ А в нашем сервисе объявим поле, куда через аннотацию @Value попросим Spring добавить значение.

```

@Service
public class TestService {

    @Value("${valueFromConfigurationFile}")
    private String str;

    @PostConstruct
    public void init() { System.out.println(str); }
}

```

Затем через init-метод распечатаем его в консоль.

▼ Запустим приложение.

```

2021-11-25 15:25:01.631 INFO 23076 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/store] : Initializing Spring embedded W
2021-11-25 15:25:01.631 INFO 23076 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: in
Hello from configuration, world!
2021-11-25 15:25:01.801 INFO 23076 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 909
2021-11-25 15:25:01.812 INFO 23076 --- [main] org.springframework.samples.store.StoreApplication : Started StoreApplication in 0

```

Как видите, текст из конфигурационного файла был помещен в переменную str нашего сервиса, а затем выведен с помощью init-метода.

Зависимость данных из конфигурации

Помимо извлечения данных из конфигурационного файла, мы можем поставить ограничение, что если этих данных в файле нет, то бин создавать нельзя.

Это делается с помощью аннотации `@ConditionalOnProperty`, которая в параметры принимает имя переменной.

▼ Попробуем закомментировать в `application.properties` нашу строку, а сервис разметить этой аннотацией.

```

#valueFromConfigurationFile=Hello from configuration, world!

```

```

@Service
@ConditionalOnProperty("valueFromConfigurationFile")
public class TestService {

    @Value("${valueFromConfigurationFile}")
    private String str;

    @PostConstruct
    public void init() { System.out.println(str); }
}

```

▼ Запустим приложение.

```

2021-11-25 15:28:06.414 INFO 17680 --- [main] pro.sky.java.itemstore.StoreApplication : Starti
2021-11-25 15:28:06.415 INFO 17680 --- [main] pro.sky.java.itemstore.StoreApplication : No act
2021-11-25 15:28:06.830 INFO 17680 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat
2021-11-25 15:28:06.834 INFO 17680 --- [main] o.apache.catalina.core.StandardService : Starti
2021-11-25 15:28:06.834 INFO 17680 --- [main] org.apache.catalina.core.StandardEngine : Starti
2021-11-25 15:28:06.872 INFO 17680 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/store] : Initia
2021-11-25 15:28:06.872 INFO 17680 --- [main] w.s.c.ServletWebServerApplicationContext : Root W
2021-11-25 15:28:07.052 INFO 17680 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat
2021-11-25 15:28:07.059 INFO 17680 --- [main] pro.sky.java.itemstore.StoreApplication : Starte

```

Ничего не было напечатано, так как бин TestService не был создан.

Аннотаций типа @ConditionalOn несколько больше, а их функционал значительно шире, чем просто анализ конфигурации.

▼ Например, они могут заблокировать создание бина в случае, если отсутствует или, наоборот, присутствует какой-то другой бин.

```

@ConditionalOn
ConditionalOnProperty org.springframework.boot.autoconfigure.condition
ConditionalOnBean org.springframework.boot.autoconfigure...
ConditionalOnClass org.springframework.boot.autoconfigure...
ConditionalOnCloudPlatform org.springframework.boot.aut...
ConditionalOnExpression org.springframework.boot.autoco...
ConditionalOnJava org.springframework.boot.autoconfigure...
ConditionalOnJndi org.springframework.boot.autoconfigure...
ConditionalOnMissingBean org.springframework.boot.autoc...
ConditionalOnMissingClass org.springframework.boot.auto...
ConditionalOnNotWebApplication org.springframework.boot...
ConditionalOnResource org.springframework.boot.autoconfigure...
ConditionalOnSingleCandidate org.springframework.boot.a...
Press Ctrl+Space to see non-imported classes Next Tip

```

Их разбирать в этом материале не будем, но по желанию можете ознакомиться и поэкспериментировать.

Выполнение методов «по-расписанию»

Продолжаем изучать базовый набор аннотаций Spring.

Spring позволяет нам выполнять некоторые методы «по расписанию».

Это осуществляется с помощью аннотации `@Scheduled`.

Эта аннотация имеет несколько параметров.

`fixedDelay`, который мы используем, позволяет указать время, которое должно проходить между вызовами этого метода.

- ▼ Перепишем наш сервис так, чтобы он выводил строку не при создании, а каждые 5 секунд. Так как `fixedDelay` принимает `ms`, нам нужно умножить количество секунд на 1000.

```
@Scheduled(fixedDelay = 5000)
public void printEvery5Sec() { System.out.println(str); }
```

Отлично, но и это еще не всё.

Чтобы Spring знал о нашем намерении выполнять какой-то код по расписанию, мы должны явно об этом сказать с помощью аннотации `@EnableScheduling`.

- ▼ Обычно эта аннотация размещается на основном классе приложения, где находится метод `main`.

```
@EnableScheduling
@SpringBootApplication
public class Application {
```

- ▼ В консоли раз в пять секунд теперь выводится наша строка.

```
2021-11-25 15:32:10.848 INFO 25360 --- [main] o.s.b.w.embedded.tomcat
Hello from configuration, world!
2021-11-25 15:32:16.855 INFO 25360 --- [main] pro.sky.java.itemstore
Hello from configuration, world!
Hello from configuration, world!
Hello from configuration, world!
```

Также аннотация `@Scheduled` принимает cron-выражения, позволяющие запускать метод, например, по четвергам в 3 часа дня.

Jackson

Мы уже познакомились с `@RestController`, который чудесным образом возвращает объект так, что в браузере отображается JSON.

И это не магия браузера.

Этим занимается библиотека Jackson, которая подтягивается вместе со Spring.

С помощью этой библиотеки мы можем конвертировать любые объекты в JSON или XML и обратно.

Для этого нужно в наш класс произвести инджекцию объекта, который называется `ObjectMapper`.

Именно он отвечает за превращение наших объектов в текст и обратно.

▼ Вернемся в наш сервис и еще немного с ним поэкспериментируем.

```
@Service
public class TestService {

    private final ObjectMapper objectMapper;

    @Contract(pure = true)
    public TestService(ObjectMapper objectMapper) { this.objectMapper = objectMapper; }

    @PostConstruct
    public void init() throws JsonProcessingException {
        Employee employee = new Employee( firstName: "Ivan", lastName: "Ivanov", salary: 100500);
        printEmployeeAsJSON( employee: employee);
    }

    public void printEmployeeAsJSON(Employee employee) throws JsonProcessingException {
        System.out.println(objectMapper.writeValueAsString( value: employee));
    }
}
```

Мы сделали инджект объекта `ObjectMapper`, затем объявили метод `printEmployeeAsJSON`, который получает в качестве параметра сотрудника и должен напечатать его в формате JSON.

Для простоты эксперимента всю логику вызова напишем в `init`-методе, но в проектах так никогда не делайте.

Задача init-метода в инициализации объекта и подготовке его к работе, а не для замены методу main.

Так как метод `writeValueAsString` выбрасывает проверяемое исключение, напомним throws.

Вообще его лучше было бы обработать, но сейчас такой задачи не стоит.

- ▼ По логике нашего кода в консоль должно быть выведено сообщение в формате JSON с теми параметрами, что мы передали в конструктор.

```
2021-11-25 15:43:58.164 INFO 1204 --- [main] w.s.c.ServletWeb
{"firstName":"Ivan","lastName":"Ivanov","salary":100500}
2021-11-25 15:43:58.384 INFO 1204 --- [main] o.s.b.w.embedded
```

И оно действительно было выведено.

Чтобы прочитать из строки объект, нужно вызвать вместо `writeValueAsString` метод `readValue`, куда передать строку и класс, который мы хотим собрать на основе строки.

- ▼ Перепишем наш метод так, чтобы он произвел обратную трансформацию.

```
@PostConstruct
public void init() throws JsonProcessingException {
    Employee employee = objectMapper.readValue(content, Employee.class);
    System.out.println(employee);
}
```

- ▼ В консоли увидим следующее.

```
2021-11-25 15:50:02.664 INFO 16880 --- [main] w.s.c.ServletWeb
Employee{firstName='Ivan', lastName='Ivanov', salary=100500}
2021-11-25 15:50:02.884 INFO 16880 --- [main] o.s.b.w.embedded
```

Это значит, что объект корректно собран на основе JSON.

Обработка исключений

Перейдем к обработке исключений, которые выбрасывают наши доблестные сервисы, они по лесенке вверх доходят до контроллера, а затем Spring их отлавливает.

Но что делать, если у нас есть какая-то особенная обработка определенного типа исключений и мы не хотим получать стену текста в консоль

от стандартной обработки Spring? Не будем ведь мы копипастить try-catch в каждый метод контроллера, переписывая необходимые действия в блоке catch?

Вот что мы хотим сделать.

Если наш контроллер выбрасывает (сам или с помощью сервисов) RuntimeException, мы хотим в консоль написать "Hola! RuntimeException is here!", а если IllegalArgumentException, то "Hallo! IllegalArgumentException is here!".

Для этого мы напишем свой ExceptionHandler.

В этом нам пригодится аннотация @ControllerAdvice.

- ▼ С помощью нее мы разметим класс, который должен заниматься подобной грязной работой, а его методы получает по аннотации @ExceptionHandler, принимающей в качестве параметра класс исключения.

```
@ControllerAdvice
public class MyExceptionHandler {

    @ExceptionHandler(RuntimeException.class)
    public void handleRuntimeException(RuntimeException e) {
        System.out.println("Hola! RuntimeException is here!");
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public void handleIllegalArgumentException(IllegalArgumentException e) {
        System.out.println("Hallo! IllegalArgumentException is here!");
    }
}
```

Протестируем.

- ▼ Создадим простой контроллер, который будет делать нам плохо, а именно выбрасывать исключения при обращении к нашему приложению.

```

@RestController
public class ExceptionController {
    @GetMapping("/runtime")
    public Object throwRuntimeException() {
        throw new RuntimeException();
    }

    @GetMapping("/illegalargument")
    public Object throwIllegalArgumentException() {
        throw new IllegalArgumentException();
    }
}

```

▼ Обратимся для примера по localhost:8080/illegalargument.

```

2021-11-25 16:08:15.698 INFO 22896 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
Hallo! IllegalArgumentException is here!

```

Наша обработка сработала, и мы получили информационное сообщение в консоль.

Как вы могли заметить, в параметрах методов нашего хэндлера находятся те исключения, что были выброшены.

И вы через переменную `e` можете обратиться к ним, получить ошибку, причину или просто распечатать стандартный стек-трейс с помощью метода `e.printStackTrace()`.